



인하공업전문대학
INHA TECHNICAL COLLEGE

C/C++ 프로그래밍

2주차

인하공업전문대학 컴퓨터정보과
김한결



목차

3.1 변수

3.2 자료형

3.3 상수

3.4 자료형 변환

학습목표

- C 언어에서 쓰이는 변수와 자료형에 대해 알아본다.
- C 언어에서 쓰이는 자료형의 크기와 범위를 이해한다.
- C 언어에서 쓰이는 상수의 개념을 이해한다.
- 자료형을 변환하는 방법을 익힌다.

Section 3.1

변수

1. 변수

■ 변수의 개념

- 컴퓨터 프로그램이 실행되는 동안 데이터를 저장하고 그 데이터를 참조하거나 수정하기 위해 사용하는 식별자
- 변수를 선언하면 메모리에 공간이 할당되고 해당 메모리 위치에 데이터 저장

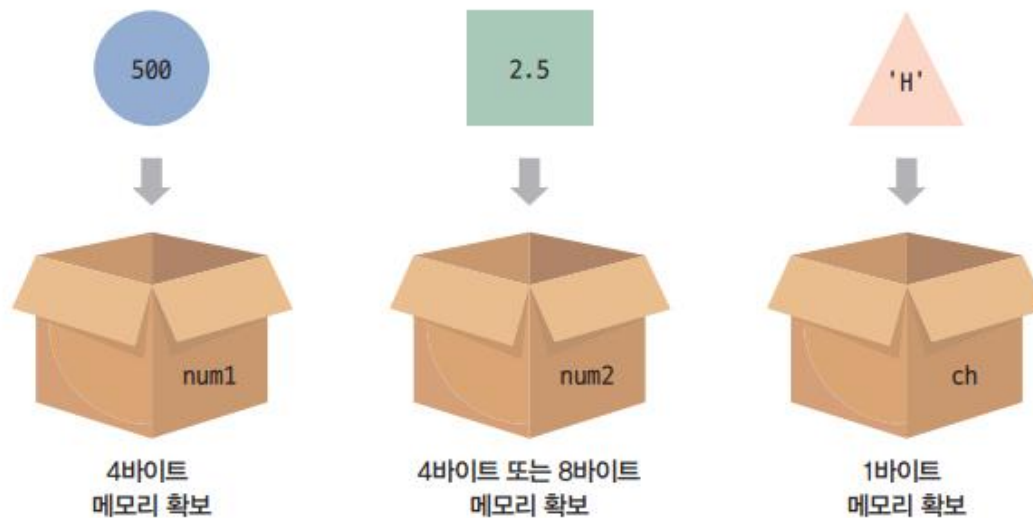
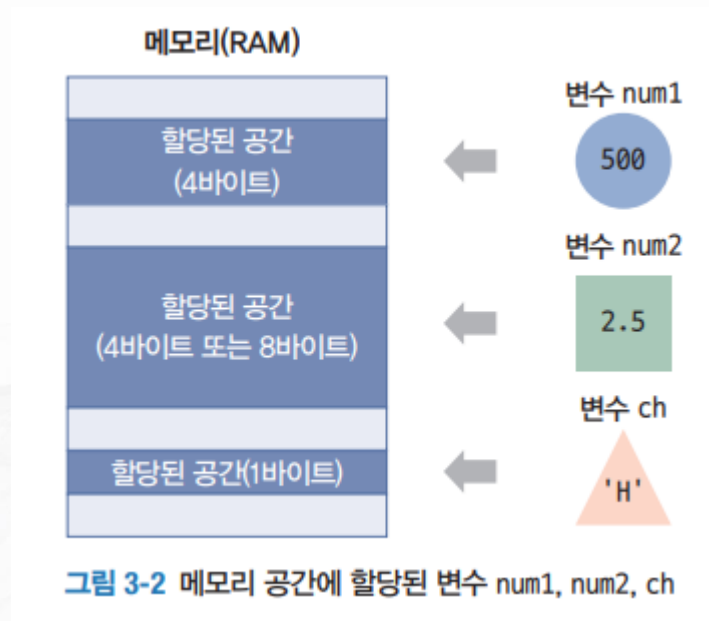


그림 3-1 선언된 변수와 데이터 크기에 따른 메모리 공간

1. 변수

■ 변수의 개념

- 선언된 변수는 자료형의 크기만큼 RAM에 공간을 할당받고, 해당 변수의 값 저장



1. 변수

■ 변수의 사용

■ 변수 선언

- 변수의 이름과 자료형을 지정하여 변수를 생성하는 과정

| Syntax | 변수 선언

자료형 변수명;
`int var;`

```
int myVariable;    // 정수형 변수 myVariable 선언  
myVariable = 17;   // 변수에 정수 17 대입
```

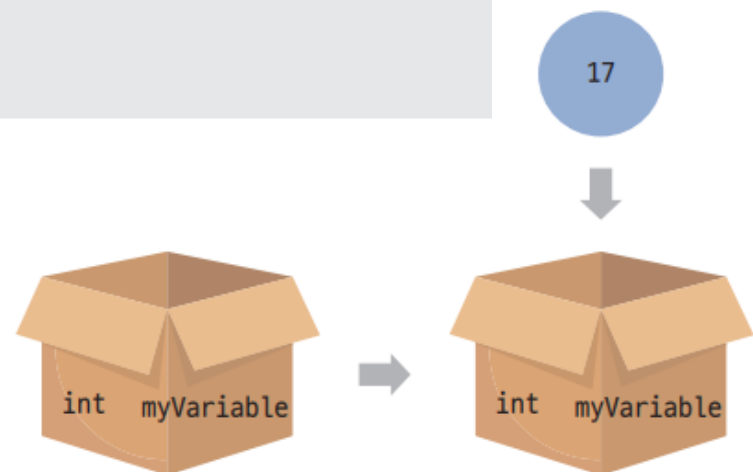


그림 3-3 변수 선언과 값 대입의 개념

1. 변수

■ 변수의 사용

■ 변수 초기화

- 생성한 변수에 초기값을 할당하는 과정

| Syntax | 변수 초기화

자료형 변수명 = 값;

`int xNum = 0;`

4바이트 크기, int형 변수값을 0으로 초기화

`double yNum = 3.3;`

8바이트 크기, double형 변수값을 3.3으로 초기화

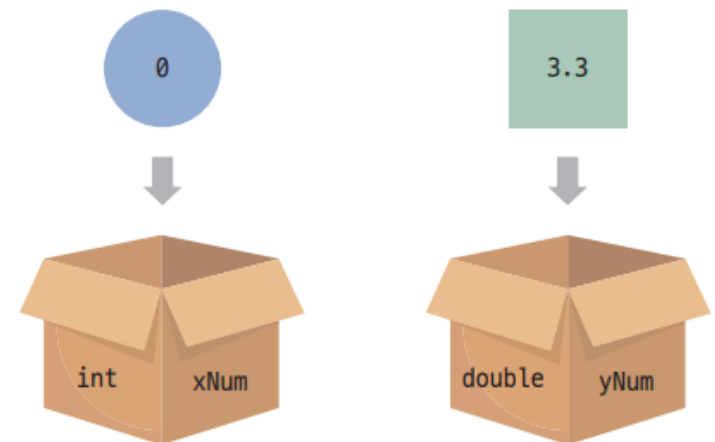


그림 3-4 변수 초기화

1. 변수

■ 변수의 사용

■ 변수명 작성

- 영문자(대·소문자), 숫자, 언더바(_) 사용 가능
- 변수명의 첫 글자는 숫자로 시작 불가능
- 대문자와 소문자 구분
- 변수명 중간에 공백 사용 불가능
- C 언어의 예약어 사용 불가능

```
countValue, student_code, _personName, DATETIME, ... // 올바른 변수명
15_value, #Test, sum value, ...                      // 잘못된 변수명
Master, master, MASTER, mAsTer                      // 각기 다른 변수
```

1. 변수

■ 변수의 사용

■ 변수를 사용하는 이유

- 값을 저장하는 메모리 공간으로 사용
- 데이터 유지와 재사용 가능
- 효율적인 계산과 연산 가능

변수를 사용한 간단한 연산

variable.c

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a, b, tot;
06
07     a = 200;
08     b = 100;
09     tot = a + b;
10
11     printf("%d", tot); // tot 출력
12
13     return 0;
14 }
```

정수형 변수 a, b, tot 선언

a의 초깃값에 200 대입

b의 초깃값에 100 대입

300

1. 변수

■ 변수의 사용

■ 변수를 사용하는 이유

- 프로그램 상태 표현에 사용

```
int pacman_hp = 25000;    // 팩맨의 생명력값 저장  
float ghost_locX = 120.2f; // 유령의 x 좌표값 저장  
float ghost_locY = 150.5f; // 유령의 y 좌표값 저장
```

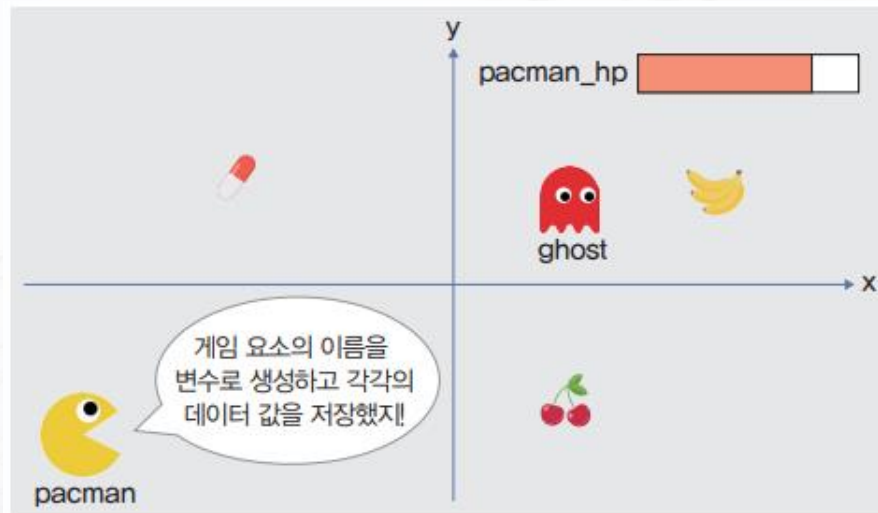


그림 3-5 변수를 활용한 게임 프로그램의 상태 표현

1. 변수

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① 변수 초기화는 변수를 선언하면서 ()을 변수에 할당하는 것을 의미한다.
- ② 어떤 프로그램의 변수가 메모리를 사용했는지 모르기 때문에 메모리가 깨끗하게 정리되지 않아 ()이 저장되어 있을 수도 있다.
- ③ 변수명 관례 중 ()은 각 단어의 첫 글자를 대문자 또는 소문자로 표기하고, ()은 각 단어를 언더바로 구분한다.

2 정수를 저장하는 변수 number를 선언하는 코드를 작성하시오.

Section 3.2

자료형

2. 자료형

■ 자료형의 개념

- 변수에 데이터를 저장할 때 데이터의 종류와 크기를 정의하는 단위

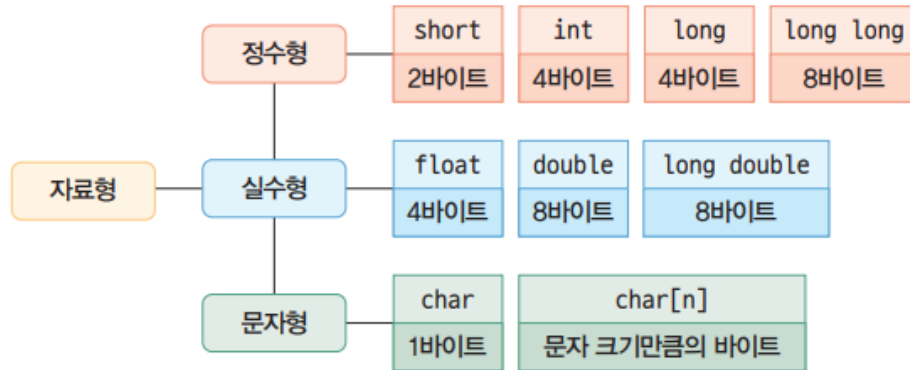


그림 3-6 자료형별 바이트 단위



그림 3-7 데이터의 유형과 크기에 맞는 자료형 사용

2. 자료형

■ 자료형의 종류

표 3-1 C 언어에서 제공하는 자료형의 종류

자료형		설명
정수형	short	int보다 작은 범위의 정수를 표현하는 데 사용된다.
	int	정수를 표현하는 데 사용되는 가장 일반적인 정수형이다.
	long	int보다 큰 범위의 정수를 표현하는 데 사용된다.
	long long	long보다 큰 범위의 정수를 표현하는 데 사용된다. 매우 큰 수를 다루거나 정밀한 연산을 할 때 유용하다.
실수형	float	단정밀도 부동 소수점 수를 표현하는 데 사용된다.
	double	배정밀도 부동 소수점 수를 표현하는 데 사용된다.
	long double	배정밀도 부동 소수점 수를 표현하는 데 사용된다. C 표준에서는 double의 최소 크기를 정의하고 있으며, long double은 그 이상의 크기와 정밀도를 나타낸다.
문자형	char	문자 1개를 표현하는 데 사용된다.
	char[n]	문자 n개를 표현하는 데 사용된다.

2. 자료형

하나 더 알기 변수 선언 시 자료형 범위의 오류

변수를 선언할 때 자료형과 저장되는 값의 범위가 맞지 않으면 메모리 공간을 할당할 때 오류가 발생한다. 하지만 비주얼 스튜디오 2022와 같은 에디터에서는 자료형 범위에 벗어난 값을 정의하면 빨간색 밑줄과 함께 오류 내용이 나타난다.

```
// 값보다 큰 범위 선언 시 오류 발생  
int a = 1657820170803929857060496847329
```

정수 상수가 너무 큼니다.

[온라인 검색](#)

그림 3-8 비주얼 스튜디오 2022의 오류 표시

2. 자료형

■ 정수형

▪ 정수형 종류와 유효 범위

- 정수형: 정수(양수, 0, 음수)를 표현하기 위한 자료형

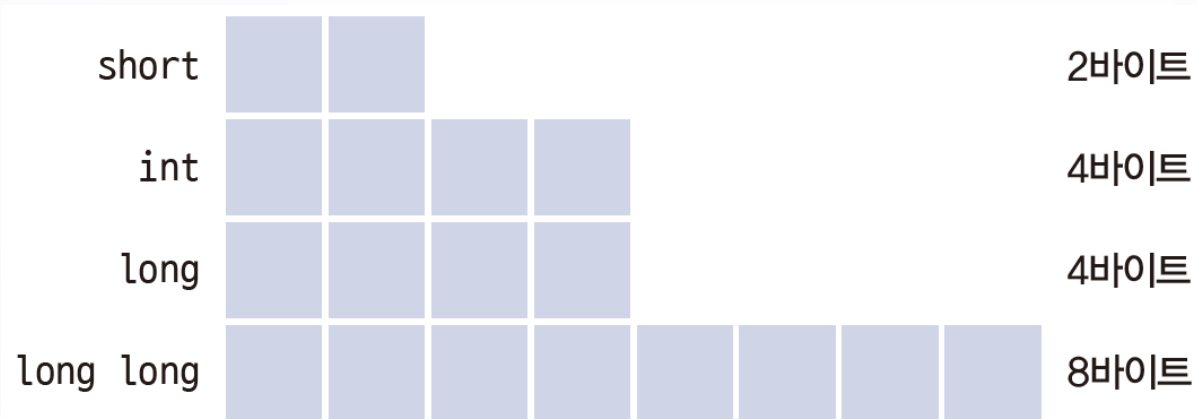


그림 3-9 정수형의 종류와 메모리 저장 공간

2. 자료형

■ 정수형

■ 정수형 종류와 유효 범위

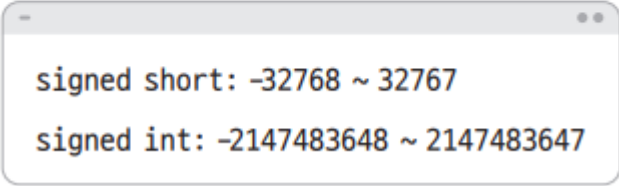
표 3-2 정수형의 종류와 유효 범위

정수형		크기	유효 범위
short	signed short	2바이트	-32,768 ~ 32,767
	unsigned short		0 ~ 65,535
int	signed int	4바이트	-2,147,483,648 ~ 2,147,483,647
	unsigned int		0 ~ 4,294,967,295
long	signed long	4바이트	-2,147,483,648 ~ 2,147,483,647
	unsigned long		0 ~ 4,294,967,295
long long	signed long long	8바이트	-9,223,372,036,854,775,808 ~ 9,223,372,036,854,775,807
	unsigned long long		0 ~ 18,446,744,073,709,551,615

2. 자료형

예제 3-1 정수형 변수를 선언하고 값 출력하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     signed short smallest_short = -32768;
06     signed short largest_short = 32767;
07
08     signed int smallest_int = -2147483648;
09     signed int largest_int = 2147483647;
10
11     printf("signed short: %d ~ %d\n", smallest_short, largest_short);
12     printf("signed int: %d ~ %d\n", smallest_int, largest_int);
13
14     return 0;
15
16 }
```



```
signed short: -32768 ~ 32767
signed int: -2147483648 ~ 2147483647
```

2. 자료형

■ 정수형

▪ sizeof 연산자

- 주어진 변수나 자료형의 크기를 바이트 단위로 알림
- sizeof 연산자로 확인한 크기는 컴파일 시 확정

| Syntax | sizeof 연산자

sizeof (피연산자); 자료형, 변수, 상수도 사용 가능

sizeof 변수명; 자료형 사용 불가

2. 자료형

예제 3-2 sizeof 연산자를 사용하여 정수형의 크기 출력하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     short sh = 12;        // short 정수형 변수
06     int nt = 155;         // int 정수형 변수
07     long long on = 1666;  // long long 정수형 변수
08
09     printf("자료형의 크기를 알아보는 코드 \n");
10     printf("1. short : %dbyte, %dbyte \n", sizeof(sh), sizeof sh);
11     printf("2. int : %dbyte, %dbyte \n", sizeof(nt), sizeof nt);
12     printf("3. long long : %dbyte, %dbyte \n", sizeof(on), sizeof on);
13
14     return 0;
15 }
```

자료형의 크기를 알아보는 코드

```
1. short : 2byte, 2byte
2. int : 4byte, 4byte
3. long long : 8byte, 8byte
```

2. 자료형

■ 정수형

▪ 정수형의 부호



그림 3-11 부호 표시 여부에 따라 달라지는 int형

2. 자료형

■ 정수형

▪ 부호 있는 정수 signed

- 부호 있는 정수를 나타내며 양수, 0, 음수를 포함하는 모든 정숫값 저장 가능

$-2^{31}, \dots, -1, 0, 1, \dots, 2^{31}-1$
 $-2,147,483,648 \sim +2,147,483,647$

▪ 부호 없는 정수 unsigned

- 부호 없는 정수를 나타내며 0과 양수만 저장 가능

$0, 1, 2, \dots, 2^{32}-1$
 $0 \sim +4,294,967,295$

```
signed int myNumber = -1024;      // 부호 있는 정수
unsigned int positiveNumber = 1024; // 부호 없는 정수
```

2. 자료형

■ 정수형

■ 오버플로

- 선언된 변수 자료형의 범위를 초과하는 값을 저장할 때 발생하는 현상



■ 언더플로

- 변수로 선언한 자료형의 유효 범위보다 더 작은 값이 대입될 때 발생하는 현상

2. 자료형

예제 3-3 int 자료형에서 오버플로가 발생하는 경우

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int maxInt = 2147483647;    // int의 최댓값으로 초기화
06     int overflow1 = maxInt + 1;
07                                     오버플로가 발생하는 연산
08     printf("maxInt: %d\n", maxInt);
09     printf("overflow1: %d\n", overflow1);
10
11     unsigned int maxUInt = 4294967295U;    // unsigned int의 최댓값
12     unsigned int overflow2 = maxUInt + 1;
13                                     오버플로가 발생하는 연산
14     printf("maxUInt: %u\n", maxUInt);
15     printf("overflow2: %u\n", overflow2);
16
17     return 0;
18 }
```

```
maxInt: 2147483647
overflow1: -2147483648
maxUInt: 4294967295
overflow2: 0
```

2. 자료형

예제 3-4 int 자료형에서 언더플로가 발생하는 경우

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int minInt1 = -2147483648LL;    // int의 최솟값으로 초기화
06     int underflow1 = minInt1 - 1;
07
08     printf("minInt: %d\n", minInt1);
09     printf("underflow1: %d\n", underflow1);
10
11     unsigned int minInt2 = 0;      // 부호 없는 int 0으로 초기화
12     unsigned int underflow2 = minInt2 - 1;
13
14     printf("minInt: %d\n", minInt2);
15     printf("underflow2: %u\n", underflow2);
16
17     return 0;
18 }
```

언더플로가 발생하는 연산

언더플로가 발생하는 연산

```
minInt: -2147483648
underflow1: 2147483647
minInt: 0
underflow2: 4294967295
```

2. 자료형

하나 더 알기 롤오버

C 언어에서 롤오버는 특정한 범위의 한계를 초과하여 다시 시작점으로 돌아가는 현상을 의미한다. 주로 정수형의 오버플로 또는 언더플로 상황에서 발생한다.

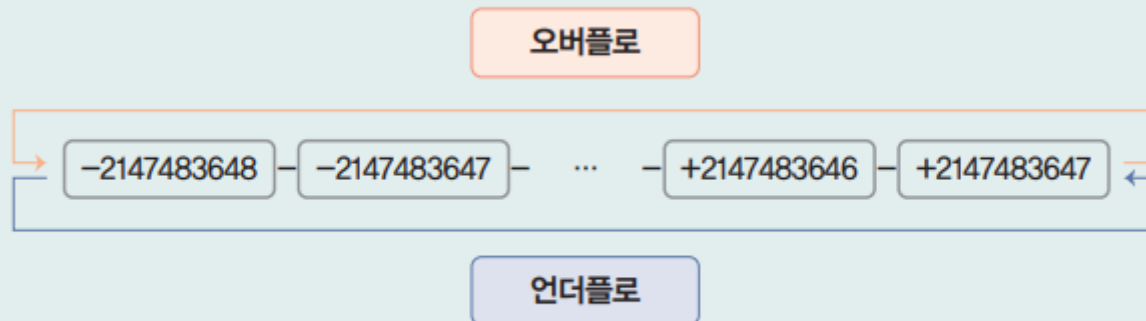


그림 3-14 `int` 자료형의 롤오버 작동 방식

2. 자료형

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① 2바이트 정수형 short에는 () short와 () short가 있다.
- ② 변수의 자료형을 long long으로 선언했을 때 할당되는 메모리 저장 공간의 크기는 () 이다.

2 다음 설명에 해당하는 것을 고르시오.

C 언어에서는 변수에 선언한 자료형이 정상적으로 처리하는 유효 범위의 최솟값보다 더 작은 값이 대입되면 이 현상이 발생한다.

- ① 롤오버 ② 오버플로 ③ 언더플로 ④ signed int

2. 자료형

■ 실수형

■ 실수형의 종류와 유효 범위

- **실수형:** 소수점이 있는 수를 저장하기 위한 자료형
- **부동 소수점 방식:** 소수점의 위치를 고정하지 않고, 가수와 실수를 이용하여 실수를 표현하는 방식

float		4바이트(정밀도 6~7자리)
double		8바이트(정밀도 15~16자리)
long double		8바이트 이상(정밀도 18자리 이상)

그림 3-15 실수형의 종류와 메모리 저장 공간

2. 자료형

■ 실수형

■ 실수형의 종류와 유효 범위

표 3-3 실수형의 종류와 정밀도, 유효 범위

실수형	크기	정밀도(유효 숫자)	유효 범위
float	4바이트	6~7자리	최솟값: $1.175494351E-38$
			최댓값: $3.402823466E+38$
double	8바이트	15~16자리	최솟값: $\pm 2.2250738585072014E-308$
			최댓값: $\pm 1.7976931348623157E+308$
long double	8바이트 이상	18자리 이상	최솟값: $\pm 3.36210314311209350626E-4932$
			최댓값: $\pm 1.18973149535723176502E+4932$

2. 자료형

예제 3-5 실수형 변수를 사용하여 사칙 연산하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     float floatValue1 = 3.14f;    // float형 변수 선언과 값 할당
06     float floatValue2 = 2.718f;    // float형 변수 선언과 값 할당
07     float sumFloat = floatValue1 + floatValue2;
08     float productFloat = floatValue1 * floatValue2;
09
10     printf("Sum (Float): %f\n", sumFloat);
11     printf("Product (Float): %f\n", productFloat);
12
13     double doubleValue1 = 123.456;    // double형 변수 선언과 값 할당
14     double doubleValue2 = 789.012;    // double형 변수 선언과 값 할당
15     double sumDouble = doubleValue1 + doubleValue2;
16     double productDouble = doubleValue1 * doubleValue2;
17
18     printf("Sum (Double): %lf\n", sumDouble);
19     printf("Product (Double): %lf\n", productDouble);
20
21     return 0;
22 }
```

변수 sumFloat에 두 변수의 덧셈 결과 저장

변수 productFloat에 두 변수의 곱셈 결과 저장

변수 sumDouble에 두 변수의 덧셈 결과 저장

변수 productDouble에 두 변수의 곱셈 결과 저장

```
Sum (Float): 5.858000
Product (Float): 8.534520
Sum (Double): 912.468000
Product (Double): 97408.265472
```

2. 자료형

예제 3-6 실수형 변수를 사용하여 삼각형과 사각형의 넓이 계산하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     float base = 5.0;
06     float height = 3.0;
07     float triangleArea = (base * height) / 2.0;
08
09     printf("삼각형 넓이: %.2f\n", triangleArea);
10
11     double width = 4.0;
12     double rheight = 6.0;
13     double rectangleArea = width * rheight;
14
15     printf("사각형 넓이: %.2lf\n", rectangleArea);
16
17     return 0;
18 }
```

삼각형의 넓이를 계산하는 함수

삼각형의 넓이 출력

사각형의 넓이를 계산하는 함수

사각형의 넓이 출력

```
삼각형 넓이: 7.50
사각형 넓이: 24.00
```

2. 자료형

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① ()는 단정밀도 부동 소수점 수를 저장하는 데 사용되는 자료형이다.
- ② ()은 배정밀도 부동 소수점 수를 저장하는 데 사용되는 자료형이다.
- ③ ()은 double형보다 더 큰 범위와 정밀도의 부동 소수점 수를 저장하는 데 사용되는 자료형이다.

2. 자료형

■ 문자형

- 단일 문자를 저장하기 위한 자료형



그림 3-16 문자형의 종류

표 3-4 문자형의 종류와 유효 범위

문자형		크기	유효 범위
char	signed char	1바이트	-128~127
	unsigned char		0~255

```
char myChar;    // 문자형 변수 선언
myChar = 'A';   // 문자 'A' 저장
```

```
char myChar;    // 문자형 변수 선언
myChar = 65;    // 아스키코드에서 'A'의 값
```

2. 자료형

■ 문자형

■ 아스키코드

- C 언어가 처음 개발되었을 당시에 주로 사용되던 문자 인코딩 체계 중 하나
- C 언어와 아스키코드가 공존하면서 발전했기에 C 언어의 문자와 문자열은 아스키코드로 변환되며, 현재 다른 언어들은 유니코드를 지원

제어 문자			출력 가능 문자									제어 문자			출력 가능 문자										
10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자	10진수	16진수	문자		
000	00	NULL	032	20	SP	064	40	@	096	60	`	016	10	DLE	048	30	0	080	50	P	112	70	p		
001	01	SOH	033	21	!	065	41	A	097	61	a	017	11	DC1	049	31	1	081	51	Q	113	71	q		
002	02	STX	034	22	"	066	42	B	098	62	b	018	12	DC2	050	32	2	082	52	R	114	72	r		
003	03	ETX	035	23	#	067	43	C	099	63	c	019	13	DC3	051	33	3	083	53	S	115	73	s		
004	04	EOT	036	24	\$	068	44	D	100	64	d	020	14	DC4	052	34	4	084	54	T	116	74	t		
005	05	ENQ	037	25	%	069	45	E	101	65	e	021	15	NAK	053	35	5	085	55	U	117	75	u		
006	06	ACK	038	26	&	070	46	F	102	66	f	022	16	SYN	054	36	6	086	56	V	118	76	v		
007	07	BEL	039	27	'	071	47	G	103	67	g	023	17	ETB	055	37	7	087	57	W	119	77	w		
008	08	BS	040	28	(	072	48	H	104	68	h	024	18	CAN	056	38	8	088	58	X	120	78	x		
009	09	HT	041	29	)	073	49	I	105	69	i	025	19	EM	057	39	9	089	59	Y	121	79	y		
010	0A	LF	042	2A	*	074	4A	J	106	6A	j	026	1A	SUB	058	3A	:	090	5A	Z	122	7A	z		
011	0B	VT	043	2B	+	075	4B	K	107	6B	k	027	1B	ESC	059	3B	;	091	5B	[	123	7B	{		
012	0C	FF	044	2C	,	076	4C	L	108	6C	l	028	1C	FS	060	3C	<	092	5C	\	124	7C			
013	0D	CR	045	2D	-	077	4D	M	109	6D	m	029	1D	GS	061	3D	=	093	5D	]	125	7D	}		
014	0E	SOH	046	2E	.	078	4E	N	110	6E	n	030	1E	RS	062	3E	>	094	5E	^	126	7E	~		
015	0F	SI	047	2F	/	079	4F	O	111	6F	o	031	1F	US	063	3F	?	095	5F	_	127	7F	DEL		

2. 자료형

예제 3-7 알파벳의 아스키코드 값 출력하기

```
01 #include <stdio.h>
```

```
02
```

```
03 int main()
```

```
04 {
```

```
05     char alphabet = 'T';
```

문자형 변수 alphabet에 'T' 저장

```
06
```

```
07     printf("alphabet 변수의 값: %c\n", alphabet);
```

변수 alphabet의 값 출력

```
08     printf("알파벳 'T'에 해당하는 ASCII 코드 값: %d\n", alphabet);
```

알파벳 'T'에 해당하는
아스키코드 값 출력

```
09
```

```
10     return 0;
```

```
11 }
```

alphabet 변수의 값: T

알파벳 'T'에 해당하는 ASCII 코드 값: 84

2. 자료형

확인문제

1 자료형의 종류 중 문자형에 대한 설명으로 옳은 것을 고르시오.

- ① 문자형은 문자열을 저장할 때 사용한다.
- ② 문자형은 부호 없는 형태로만 사용할 수 있다.
- ③ 문자형은 단일 문자를 저장할 때 사용한다.
- ④ `string`형을 사용하여 단일 문자를 변수에 저장한다.

2 다음 빈칸에 알맞은 말을 넣으시오.

- ① 아스키코드는 ()비트로 모든 알파벳을 표현할 수 있다.
- ② 아스키코드의 체계에 따르면 대문자 V를 ()으로, 소문자 ()를 118로 나타낼 수 있다.

Section 3.3

상수

3. 상수

■ 상수의 개념

- 프로그램에서 고정된 값을 나타내는 식별자
- 한 번 정의하면 그 값이 유지

- 정수형 상수

- 실수형 상수

- 문자형 상수

- 단일 문자를 나타내는 상수로, 작은따옴표(' ')로 표기

- 문자열 상수

- 여러 개의 문자로 이루어진 문자열을 나타내는 상수로, 큰따옴표("")로 표기

변수 1 상수 변수 2

원의 넓이 = 3.14 × 반지름²

그림 3-17 원의 넓이를 구하는 공식의 상수와 변수

3. 상수

■ 정수형 상수

■ 10진수

```
printf("Decimal: %d\n", 42);
```

10진수 정수를 출력하는 형식 지정자

■ 16진수

```
printf("Hexadecimal: %x\n", 0x2A);
```

16진수 정수를 출력하는 형식 지정자

■ 8진수

```
printf("Octal: %o\n", 052);
```

8진수 정수를 출력하는 형식 지정자

■ 2진수

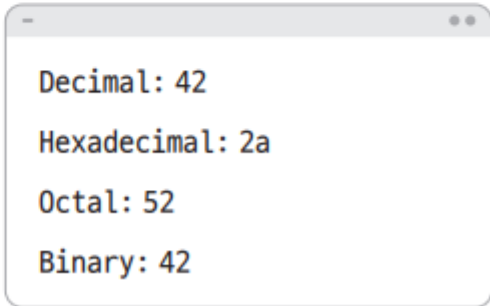
```
printf("Binary: %d\n", 0b101010);
```

10진수 정수를 출력하는 형식 지정자

3. 상수

예제 3-8 10진수, 16진수, 8진수, 2진수 상수 출력하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int decimal = 42;           // 10진수 상수
06     int hexadecimal = 0x2A;     // 16진수 상수
07     int octal = 052;           // 8진수 상수
08     int binary = 0b101010;     // 2진수 상수
09
10     printf("Decimal: %d\n", decimal);
11     printf("Hexadecimal: %x\n", hexadecimal);
12     printf("Octal: %o\n", octal);
13     printf("Binary: %d\n", binary);
14
15     return 0;
16 }
```



```
Decimal: 42
Hexadecimal: 2a
Octal: 52
Binary: 42
```

3. 상수

■ 실수형 상수

- 부동 소수점 형식으로 표현
- **10진수 상수:** 10진수 형식으로 나타낸 실수
 - [예] 3.14, - 2.5, 0.0
- **16진수 상수:** 16진수 형식으로 나타낸 실수
 - [예] 0x1.8p2는 1.8×2^2 을 나타냄
- **지수 표기법:** e 또는 E를 사용하여 지수를 표기
 - [예] 1.23e - 5는 1.23×10^{-5} 을 나타냄

3. 상수

■ 실수형 상수

■ 지수 형식

- 가수

- 실수의 유효 숫자 부분

- 지수

- 실수를 곱하는 지수를 의미

589.734



5.89734



10²

유효 숫자 가수 부분

지수 부분

그림 3-18 지수 형식의 구성 요소



그림 3-19 지수 형식으로 나타낸 실수형 상수

3. 상수

■ 실수형 상수

▪ float형 상수

- 32비트 부동 소수점 형식으로 표현되는 실숫값

```
float fscore;
```

```
fscore = 589.734F;
```

float형 상수 표현 형식으로 f 또는 F를 사용



그림 3-20 float형 상수의 비트 자릿수 구성

3. 상수

예제 3-9 float형 상수 출력하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     float fscore;
06     fscore = 589.73456F;
07
08     printf("fscore: %f\n", fscore);
09     printf("fscore: %.2f\n", fscore);
10     printf("fscore: %e\n", fscore);
11     printf("fscore: %E\n", fscore);
12
13     return 0;
14 }
```

float형 상수 589.73456

소수점 아래 둘째 자리까지 출력

지수 형식 e로 출력

지수 형식 E로 출력

```
fscore: 589.734558
fscore: 589.73
fscore: 5.897346e+02
fscore: 5.897346E+02
```

3. 상수

■ 실수형 상수

▪ double형 상수

- 64비트 부동 소수점 형식으로 표현되는 실숫값

```
double dscore;  
dscore = 589.73456;
```



그림 3-21 double형 상수의 비트 자릿수 구성

3. 상수

예제 3-10 double형 상수 출력하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     double dscore;
06     dscore = 589.73456;
07
08     printf("dscore: %lf\n", dscore);
09     printf("dscore: %.2lf\n", dscore);
10     printf("dscore: %e\n", dscore);
11     printf("dscore: %E\n", dscore);
12
13     return 0;
14 }
```

double형 상수 589.73456

소수점 아래 여섯째 자리까지 출력

소수점 아래 두 자리 정밀도로 double 값 출력

지수 형식 e로 출력

지수 형식 E로 출력

```
dscore: 589.734560
dscore: 589.73
dscore: 5.897346e+02
dscore: 5.897346E+02
```

3. 상수

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① float형 상수는 () 형식으로 표현되는 실숫값이다.
- ② double형 상수는 () 형식으로 표현되는 실숫값이다.

3. 상수

■ 문자형 상수

- **알파벳 대문자:** 알파벳 대문자를 나타내는 문자형 상수
 - [예] 'A', 'B', 'C'
- **알파벳 소문자:** 알파벳 소문자를 나타내는 문자형 상수
 - [예] 'a', 'b', 'c'
- **숫자:** 숫자를 나타내는 문자형 상수
 - [예] '0', '1', '2'
- **특수 문자:** 특수 문자를 나타내는 문자형 상수
 - [예] '\$', '%', '@'

3. 상수

예제 3-11 문자형 상수를 아스키코드로 출력하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     char grade = 'A';
06     char dollar = '$';
07     char digit = '7';
08
09     printf("Grade 값: %c\n", grade);
10     printf("Grade 아스키코드: %d\n", grade);
11
12     printf("Dollar Sign 값: %c\n", dollar);
13     printf("Dollar Sign 아스키코드: %d\n", dollar);
14
15     printf("Digit 값: %c\n", digit);
16     printf("Digit 아스키코드: %d\n", digit);
17
18     return 0;
19 }
```

문자를 출력하는 형식 지정자

문자 '7'에 해당하는 아스키코드 값 출력

```
Grade 값: A
Grade 아스키코드: 65
Dollar Sign 값: $
Dollar Sign 아스키코드: 36
Digit 값: 7
Digit 아스키코드: 55
```

3. 상수

■ 기호 상수

- 반복적으로 사용하는 값이나 매직 넘버를 대체하기 위해 이름으로 정의된 상수
- 전역상수로 선언되며, 일반적으로 대문자와 언더바(_)를 사용하여 이름 작성

■ #define 지시문을 사용하여 정의

| Syntax | #define 지시문

지시문 기호 상수 상숫값

```
#define PI 3.14159
```

■ const 명령어를 사용하여 정의

| Syntax | const 명령어

명령어 자료형 기호 상수 상숫값;

```
const double PI 3.14159;
```

3. 상수

하나 더 알기 매직 넘버

C 언어에서 매직 넘버(magic number)는 소스 코드 내에서 문맥 없이 사용된 특정 숫자를 의미한다. 이러한 숫자는 코드의 명확성을 감소시키며, 다른 사람들이 코드를 읽을 때 그 숫자가 무엇을 의미하는지, 왜 그러한 값이 선택되었는지 알기 어렵다. 다음 코드를 살펴보자.

```
if (salary > 40000)
{
    // do something
}
```

매직 넘버

여기서 숫자 '40000'은 매직 넘버로, 코드만 보아서는 이 숫자의 의미가 명확하지 않다. 이럴 때 다음과 같이 UPPER_LIMIT_SALARY라는 기호 상수를 사용하면 해당 숫자가 최대 급여 한도를 의미한다는 것을 쉽게 알 수 있다.

```
const int UPPER_LIMIT_SALARY = 40000;
```

기호 상수를 정의하는 구문

```
if (salary > UPPER_LIMIT_SALARY)
{
    // do something
}
```

숫자 대신 기호 상수 입력

이처럼 매직 넘버 대신 의미 있는 이름의 상수나 변수를 사용하면 코드를 직관적으로 이해할 수 있어 중요한 값이 변경되는 것을 방지함으로써 오류가 발생할 가능성도 줄어든다.

3. 상수

예제 3-12 #define 지시문으로 기호 상수 정의하고 활용하기

```
01 #include <stdio.h>
02
03 #define PI 3.14159265
04
05 int main()
06 {
07     float radius = 5.0f;
08     float area = 0;
09
10     area = PI * radius * radius;
11
12     printf("반지름: %.2f\n", radius);
13     printf("기호 상수 PI: %.8f\n", PI);
14     printf("원의 넓이: %.8f\n", area);
15
16     return 0;
17 }
```

#define 지시문으로 기호 상수 정의

기호 상수 PI를 이용한 연산 코드

소수점 아래 여덟째 자리까지 출력

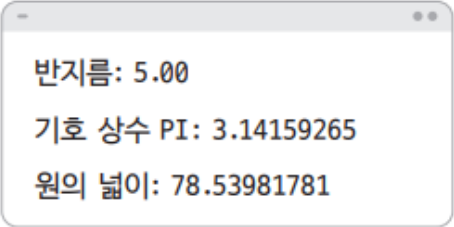
```
반지름: 5.00
기호 상수 PI: 3.14159265
원의 넓이: 78.53981781
```

3. 상수

예제 3-13 `const` 명령어로 기호 상수 정의하고 활용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     const double PI = 3.14159265;
06
07     double radius = 5.0;
08     double area = 0;
09
10     area = PI * radius * radius;
11
12     printf("반지름: %.2lf\n", radius);
13     printf("기호 상수 PI: %.8lf\n", PI);
14     printf("원의 넓이: %.8lf\n", area);
15
16     return 0;
17 }
```

`const` 명령어로 기호 상수 선언



```
반지름: 5.00
기호 상수 PI: 3.14159265
원의 넓이: 78.53981781
```

3. 상수

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① 프로그램에서 고정된 문자를 나타내는 데 사용되는 문자형 상수는 ()로 감싼다.
- ② 기호 상수를 정의하는 방법에는 () 지시문을 사용하는 방법과 () 명령어를 사용하는 방법이 있다.

Section 3.4

자료형 변환

4. 자료형 변환

■ 자료형 변환의 정의

- 어떤 자료형의 데이터를 다른 자료형으로 변환하는 과정
- C 언어에서 변수를 선언하고 프로그램을 실행하다 보면 연산 처리 과정에서 원치 않는 결괏값이 산출되는 경우
 - 예를 들어 정수와 정수의 나눗셈 연산을 수행하여 실수형 변수에 결괏값을 대입했을 때 실수가 아닌 정수가 실행 결과로 출력 가능

4. 자료형 변환

■ 자료형 변환의 정의

정수와 정수의 나눗셈 연산

Division.c

```
01 #include <stdio.h>
02
03 int main()
04 {
05
06     int a = 5;    // int형 변수 a를 선언하고 5로 초기화
07     int b = 3;    // int형 변수 b를 선언하고 3으로 초기화
08     float c = a / b;
09
10     printf("%d / %d = %f \n", a, b, c);
11
12     return 0;
13 }
```

5 / 3 = 1.000000

정수 부분만을 반환(소수점 공간이 없음)

4. 자료형 변환

■ 자동 형 변환

- 특정 연산에서 다른 자료형이 함께 사용될 때 컴파일러가 크기가 작은 자료형에서 크기가 큰 자료형으로 자동으로 변환



그림 3-22 자동 형 변환의 순서

```
int i = 10;           // int형 변수 i를 선언하고 10으로 초기화
float f = 2.5;         // float형 변수 f를 선언하고 2.5로 초기화
float result = i * f;  // i가 float형으로 자동으로 변환됨
```

- ① 먼저 int형의 값이 데이터 크기가 더 큰 float형으로 자동으로 변환된다.
- ② 따라서 두 float 값의 곱셈 연산이 수행되어 결과값이 float형으로 반환된다.

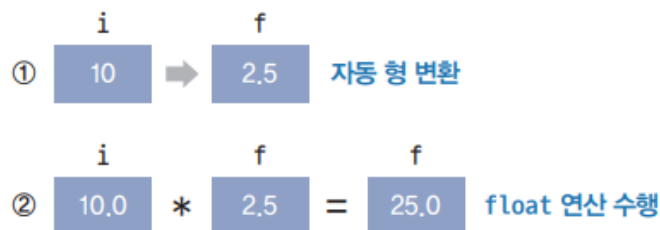


그림 3-23 자동 형 변환의 연산 과정

4. 자료형 변환

■ 명시적 형 변환

- 프로그래머가 직접 자료형 변환을 지시하는 방식

| Syntax | 형 변환 연산자

(자료형)피연산자

(int)2.78

```
int i = 10;           // int형 변수 i를 선언하고 10으로 초기화
float f = 2.5;        // float형 변수 f를 선언하고 2.5로 초기화
int result = (int)f * i; // 형 변환 연산자를 사용하여 변수 f를 int형으로 변환
```

4. 자료형 변환

예제 3-14 자동 형 변환과 명시적 형 변환으로 연산 수행하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int i = 5;
06     double d = 2.2;
07     double result1 = i + d;
08     double result2 = i * d;
09     double result3 = (double)i - d;
10     double result4 = (double)i / d;
11
12     printf("덧셈 결과: %lf\n", result1);
13     printf("곱셈 결과: %lf\n", result2);
14     printf("뺄셈 결과: %lf\n", result3);
15     printf("나눗셈 결과: %lf\n", result4);
16     return 0;
17 }
```

연산 과정에서 int형 변수 i가
double형으로 자동 형 변환

형 변환 연산자를 이용하여 int형 변수
i를 double형으로 명시적 형 변환

```
덧셈 결과: 7.200000
곱셈 결과: 11.000000
뺄셈 결과: 2.800000
나눗셈 결과: 2.272727
```

4. 자료형 변환

예제 3-15 double형에서 int형으로 자동 형 변환 수행하기

```
01 #include <stdio.h>
```

```
02
```

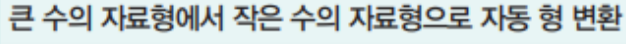
```
03 int main()
```

```
04 {
```

```
05     double d = 5.0;
```

```
06     int i;
```

```
07      double형 변수 d의 값을 int형 변수 i에 대입
```

```
08      i = d;  큰 수의 자료형에서 작은 수의 자료형으로 자동 형 변환
```

```
09
```

```
10     printf("i의 값: %d\n", i);
```

```
11
```

```
12     return 0;
```

```
13 }
```

i의 값: 5  소수 부분이 제거되고 정수 부분만 반환

4. 자료형 변환

예제 3-16 double형에서 int형으로 명시적 형 변환 수행하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     double d = 5.0;
06     int i;
07
08     i = (int)d;
09
10     printf("i의 값: %d\n", i);
11
12     return 0;
13 }
```

double형 변수 d의 값을 int형으로 명시적
형 변환을 한 후 그 결과를 변수 i에 할당

큰 수의 자료형에서 작은 수의
자료형으로 명시적 형 변환

i의 값: 5 소수 부분이 제거되고 정수 부분만 반환

4. 자료형 변환

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① ()은 C 컴파일러가 자동으로 자료형을 변환하는 방식이다.
- ② ()은 프로그래머가 직접 자료형을 변환하는 방식이다.

2 다음과 같이 명시적 형 변환을 했을 때 출력되는 i의 값은 무엇인가?

```
float f = 3.5;  
int i;  
i=(int)f;
```

- ① 0.35
- ② 3.5
- ③ 3.0
- ④ 3

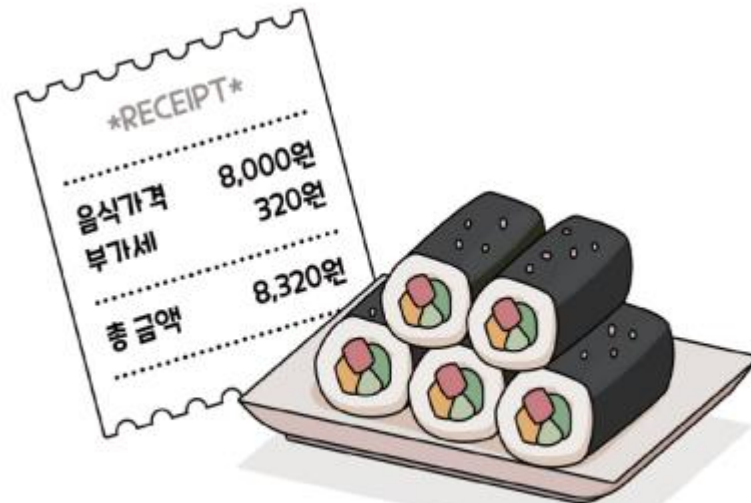
4. 자료형 변환

LAB 음식 가격에 부가세 적용하기

■ 실행 결과

음식의 가격을 입력하세요(원): 8000

부가세 포함 총가격: 8320.00원



4. 자료형 변환

LAB 음식 가격에 부가세 적용하기

■ 조건

- 음식 가격을 입력받는다.
- 입력받은 음식 가격에 4%의 부가세를 덧붙여 총가격을 계산한다.
- 부가세가 포함된 총가격을 소수점 아래 둘째 자리까지 화면에 출력한다.

■ 힌트

- 상수를 사용하여 부가세율을 0.04로 정의한다.
- 부가세가 포함된 총가격은 입력받은 음식 가격에 부가세율을 곱한 값을 더하여 계산한다.
- printf() 함수의 형식 지정자를 사용하여 소수점 아래 둘째 자리까지 출력한다.
- 음식 가격을 입력받아 변수를 선언하고, 이 변수에 부가세율을 적용하여 총가격을 계산한다. 단, 음식 가격을 입력받기 전 변수의 초기값을 확인해야 한다.

4. 자료형 변환

LAB 음식 가격에 부가세 적용하기

문제 해결

LAB_3-1.c

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03 #define VAT_RATE 0.04 // 상수 선언, 부가세율 0.04
04 // 부가세율을 나타내는 상수 정의
05 int main()
06 {
07     double foodPrice; // 음식 가격 초기화
08     double totalPrice = 0.0; // 총가격 초기화
09     // 부가세가 포함된 가격을 나타내는 변수 totalPrice 선언 및 초기화
10     printf("음식의 가격을 입력하세요(원): ");
11     scanf("%lf", &foodPrice);
12     // 부동 소수점 수를 입력받기 위한 형식 지정자
13     totalPrice = foodPrice + (foodPrice * VAT_RATE); // 총가격 계산
14
15     printf("부가세 포함 총가격: %.2lf원\n", totalPrice);
16     // 소수점 아래 둘째 자리까지 출력하는 형식 지정자
17     return 0;
18 }
```

4. 자료형 변환

LAB 전기 요금 계산하기

■ 실행 결과

```
사용한 전력량(kW)을 입력하세요: 150
전력 요금(1kW당 비용)을 입력하세요: 20
전기 요금: 3000
```



■ 조건

- 전력 사용량(kW)과 전력 요금(1kW당 비용)을 정수형으로 입력받는다.
- 전력 요금을 계산할 때 오버플로를 방지하기 위해 int형보다 큰 범위의 정수를 저장할 수 있도록 형 변환을 수행한다.

■ 힌트

- 전기 요금은 전력 사용량과 전력 요금을 곱하여 계산한다.
`long long 전기 요금 = (long long)전력 사용량 * 전력 요금;`
- `printf()` 함수의 형식 지정자 `%lld`를 사용하여 전기 요금을 출력한다.

4. 자료형 변환

LAB 전기 요금 계산하기

문제 해결

LAB_3-2.c

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int powerConsumed, costPerkW;    // 정수형 변수 선언
07
08     printf("사용한 전력량(kW)을 입력하세요: ");
09     scanf("%d", &powerConsumed);
10     printf("전력 요금(1kW당 비용)을 입력하세요: ");
11     scanf("%d", &costPerkW);
12
13     long long totalCost = (long long)powerConsumed * costPerkW;
14
15     printf("전기 요금: %lld\n", totalCost);
16
17     return 0;
18 }
```

전력 사용량과 전력 요금을 입력받아
저장하기 위한 형식 지정자 %d

오버플로를 방지하기 위한 명시적 형 변환

long long 자료형에 대한 형식 지정자 %lld

목차

4.1 연산자의 개요

4.2 산술·대입·증감 연산자

4.3 논리·조건·관계 연산자

4.4 기타 연산자

4.5 연산자의 우선순위

학습목표

- C 언어에서 쓰이는 수식과 연산자의 개념을 이해한다.
- C 언어에서 제공하는 연산자의 종류를 파악하고 사용법을 익힌다.
- 연산자의 우선순위와 결합 방향에 대해 알아본다.

Section 4.1

연산자의 개요

1. 연산자의 개요

■ 수식과 연산자

- 수식: 하나 이상의 값, 변수, 연산자를 결합하여 새로운 값을 만들어내는 구문
- C 언어의 수식은 수학에서 사용하는 수식과 비슷한 형태
- 수식은 하나의 값이나 변수가 될 수 있으며, 연산으로 계산된 결과는 다른 변수에 할당되거나 출력 가능
- 연산자와 피연산자로 구성
- 다양한 연산자를 활용하여 산술 연산, 비교 연산, 논리 연산, 대입 연산 등을 수행
 - 산술 수식: $22 * 3 = 66$
 - 관계 수식: $100 > 200 = 0$
 - 논리 수식: $(32 < 53) \&\& (12 > 6) = 1$
 - 복합 수식: $(10 + 2) * (20 - 3) = 204$

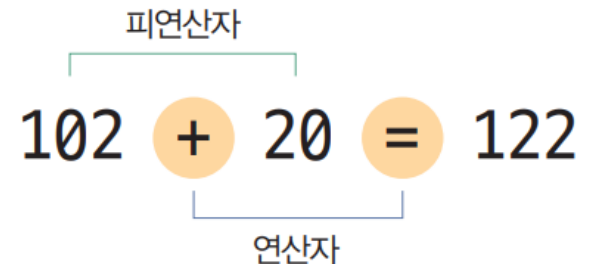


그림 4-1 수식의 기본 형태

1. 연산자의 개요

확인문제

1 다음 빈칸에 알맞은 말을 넣으시오.

- ① C 언어에서는 () 연산, () 연산, () 연산, () 연산 등을 수행할 수 있으며, 그 결과가 다른 수식이나 명령문에 사용되기도 한다.
- ② ()는 특정 연산을 수행하는 기호이다.

Section 4.2

산술·대입·증감 연산자

2. 산술·대입·증감 연산자

■ 산술 연산자

- 덧셈, 뺄셈과 같은 수학적 계산을 수행하기 위한 연산자

표 4-1 산술 연산자의 종류

산술 연산자		예	결과값
덧셈 연산자	+	<code>int sum = 5 + 3;</code>	8
뺄셈 연산자	-	<code>int difference = 5 - 3;</code>	2
곱셈 연산자	*	<code>int product = 5 * 3;</code>	15
나눗셈 연산자	/	<code>int quotient = 5 / 3;</code>	1
나머지 연산자	%	<code>int remainder = 5 % 3;</code>	2

2. 산술·대입·증감 연산자

■ 산술 연산자

- 2개 이상의 피연산자가 필요
- 피연산자의 자료형
- 연산 결과값의 자료형

```
int result = (10 * 2) / (20 - 3) // 정수형 연산은 정수형 결과값 반환  
float area = (7 * 7) * 3.14     // 실수형 연산은 실수형 결과값 반환
```

- 산술 연산자의 우선순위
- 나눗셈의 특별한 경우
- 산술 연산의 형 변환

```
int integerNumber = 5;    // 정수  
double floatNumber = 6.7; // 실수  
double result = integerNumber + floatNumber; // 정수와 실수를 더하는 연산
```

2. 산술·대입·증감 연산자

예제 4-1 산술 연산자 사용하여 계산하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 15, b = 4; // 정수형 변수 a, b 선언 및 초기화
06
07     int sum = a + b;    // 정수의 덧셈
08     int difference = a - b; // 정수의 뺄셈
09     int product = a * b; // 정수의 곱셈
10     int quotient = a / b; // 정수의 나눗셈
11
12     printf("sum: %d\n", sum);
13     printf("difference: %d\n", difference);
14     printf("product: %d\n", product);
15     printf("quotient: %d\n", quotient);
16
17     float x = 5.5, y = 2.3; // 실수형 변수 x, y 선언 및 초기화
18
19     float floatSum = x + y; // 부동 소수점 수의 덧셈
20     float floatDifference = x - y; // 부동 소수점 수의 뺄셈
21     float floatProduct = x * y; // 부동 소수점 수의 곱셈
22     float floatQuotient = x / y; // 부동 소수점 수의 나눗셈
23
24     printf("floatSum: %f\n", floatSum);
25     printf("floatDifference: %f\n", floatDifference);
26     printf("floatProduct: %f\n", floatProduct);
27     printf("floatQuotient: %f\n", floatQuotient);
28
29     return 0;
30 }
```

```
sum: 19
difference: 11
product: 60
quotient: 3
floatSum: 7.800000
floatDifference: 3.200000
floatProduct: 12.650000
floatQuotient: 2.391304
```

2. 산술·대입·증감 연산자

■ 대입 연산자

- 변수에 값을 저장할 때 사용하는 연산자

| Syntax | 대입 연산자

```
변수 = 상수;  
변수 = 수식;  
변수 = 다른 변수;
```

```
a = 25;  
a = 5 * 3.4;  
a = other;
```

```
int a, b;    // int형 변수 a와 b 선언  
a = b = 5;   // 변수 b에 5를 대입한 후 변수 a에 변수 b의 값 대입
```

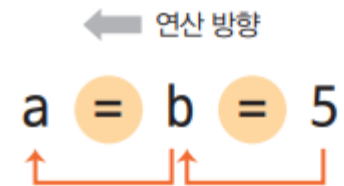


그림 4-2 대입 연산자의 연산 방향

- 대입 연산자의 기호는 =인데, 수학에서 =는 '같다'를 의미하지만, C 언어의 대입 연산자는 오른쪽의 값을 왼쪽의 변수에 할당한다는 의미

2. 산술·대입·증감 연산자

예제 4-2 대입 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 10;
06     printf("a = %d\n", a);
07
08     int x, y, z;
09     x = y = z = 20;
10     printf("x = %d, y = %d, z = %d\n", x, y, z);
11
12     int num = 5, score = 3;
13     num += 3;
14     score *= 2;
15     printf("num = %d\n", num);
16     printf("score = %d\n", score);
17
18     return 0;
19 }
```

변수 a 선언 및 초기화

a의 값 출력

변수 x, y, z 선언

동일한 값 20을 연쇄적으로 대입

```
a = 10
x = 20, y = 20, z = 20
num = 8
score = 6
```

2. 산술·대입·증감 연산자

■ 복합 대입 연산자

- 특정 연산과 대입 연산을 동시에 수행하기 위한 연산자
- 복합 대입 연산자의 몇 가지 예

```
int x = 10;    // int형 변수 x에 10 저장
x += 5;        // x의 값은 15
x *= 2;        // x의 값은 30
x /= 5;        // x의 값은 6
x %= 5;        // x의 값은 0
```

2. 산술·대입·증감 연산자

■ 복합 대입 연산자

표 4-2 복합 대입 연산자의 종류

복합 대입 연산자	설명	예
<code>+=</code>	왼쪽 피연산자에 오른쪽 피연산자를 더한 값을 왼쪽 피연산자에 대입한다.	<code>a += b;</code> <code>a = a + b;</code>
<code>-=</code>	왼쪽 피연산자에서 오른쪽 피연산자를 뺀 값을 왼쪽 피연산자에 대입한다.	<code>a -= b;</code> <code>a = a - b;</code>
<code>*=</code>	왼쪽 피연산자에 오른쪽 피연산자를 곱한 값을 왼쪽 피연산자에 대입한다.	<code>a *= b;</code> <code>a = a * b;</code>
<code>/=</code>	왼쪽 피연산자를 오른쪽 피연산자로 나눈 값을 왼쪽 피연산자에 대입한다.	<code>a /= b;</code> <code>a = a / b;</code>
<code>%=</code>	왼쪽 피연산자를 오른쪽 피연산자로 나눈 값의 나머지를 왼쪽 피연산자에 대입한다.	<code>a %= b;</code> <code>a = a % b;</code>
<code>&=</code>	두 피연산자의 비트를 AND 연산한 값을 왼쪽 피연산자에 대입한다.	<code>a &= b;</code> <code>a = a & b;</code>
<code> =</code>	두 피연산자의 비트를 OR 연산한 값을 왼쪽 피연산자에 대입한다.	<code>a = b;</code> <code>a = a b;</code>
<code>^=</code>	두 피연산자의 비트를 XOR 연산한 값을 왼쪽 피연산자에 대입한다.	<code>a ^= b;</code> <code>a = a ^ b;</code>
<code><<=</code>	왼쪽 피연산자의 비트를 오른쪽 피연산자의 값만큼 왼쪽으로 이동한 뒤 그 결과를 왼쪽 피연산자에 대입한다.	<code>a <<= b;</code> <code>a = a << b;</code>
<code>>>=</code>	왼쪽 피연산자의 비트를 오른쪽 피연산자의 값만큼 오른쪽으로 이동한 뒤 그 결과를 왼쪽 피연산자에 대입한다.	<code>a >>= b;</code> <code>a = a >> b;</code>

2. 산술·대입·증감 연산자

예제 4-3 복합 대입 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05
06
07     printf("num 변수의 초기 상태 값: %d\n", num);
08
09     num += 5;    // num = num + 5;
10     printf("num += 5 연산 후: %d\n", num);
11
12     num -= 3;    // num = num - 3;
13     printf("num -= 3 연산 후: %d\n", num);
14
15     num *= 4;    // num = num * 4;
16     printf("num *= 4 연산 후: %d\n", num);
17
18     num /= 2;    // num = num / 2;
19     printf("num /= 2 연산 후: %d\n", num);
20
21     num %= 7;    // num = num % 7;
22     printf("num %= 7 연산 후: %d\n", num);
23
24     return 0;
25 }
```

num 변수의 초기 상태 값: 10

num += 5 연산 후: 15

num -= 3 연산 후: 12

num *= 4 연산 후: 48

num /= 2 연산 후: 24

num %= 7 연산 후: 3

누적 연산에 따른 num 값의 변화

2. 산술·대입·증감 연산자

■ 증감 연산자

- 변수의 값을 1씩 증가시키거나 감소시키는 연산자

표 4-3 증감 연산자의 종류

증감 연산자		설명	예
증가 연산자	++	변수의 값을 1만큼 증가시킨다.	a++ ++a
감소 연산자	--	변수의 값을 1만큼 감소시킨다.	a-- --a

2. 산술·대입·증감 연산자

■ 증감 연산자

■ 전위 증감 연산자

- 연산자 다음에 오는 변수의 값을 즉시 증가시키거나 감소

```
i = 1;  
num = ++i;  
// 전위 증가 연산자
```

i 값을 1 증가시킨 후 num에 i 값 대입

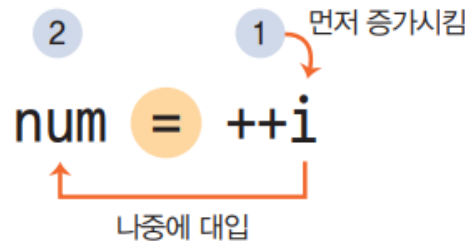


그림 4-3 전위 증감 연산자의 연산 방식

2. 산술·대입·증감 연산자

■ 증감 연산자

▪ 후위 증감 연산자

- 현재 변수값을 먼저 반환하고 나서 변수값을 증가시키거나 감소

```
i = 1;
num = i++;
```

num에 i 값을 대입한 후 i 값을 1 증가시킴
// 후위 증가 연산자

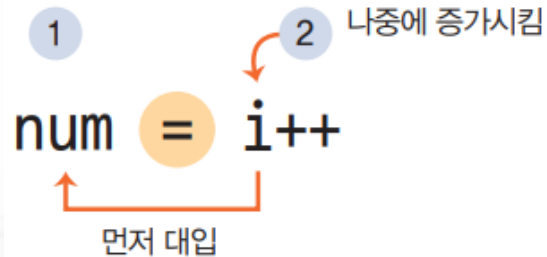


그림 4-4 후위 증감 연산자의 연산 방식

```
i = 1;
num = --i;
```

i 값을 1 감소시킨 후 num에 i 값 대입
// 전위 감소 연산자

```
i = 1;
num = i--;
```

num에 i 값을 대입한 후 i 값을 1 감소시킴
// 후위 감소 연산자

2. 산술·대입·증감 연산자

예제 4-4 증감 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 5, b = 5;
06
07     printf("초기상태 a: %d\n", a);
08     printf("초기상태 b: %d\n", b);
09
10     printf("전위 연산자 사용 후, a: %d\n", ++a);
11     printf("후위 연산자 사용 후, b: %d\n", b++);
12
13     printf("연산 후 결과 a: %d\n", a);
14     printf("연산 후 결과 b: %d\n", b);
15
16     return 0;
17 }
```

int형 변수 선언 및 초기화

변수 a, b의 값 출력

변수 a의 값을 1 증가시킨 후 출력

변수 b의 값을 출력한 후 b의 값을 1 증가시킴

```
초기상태 a: 5
초기상태 b: 5
전위 연산자 사용 후, a: 6
후위 연산자 사용 후, b: 5
연산 후 결과 a: 6
연산 후 결과 b: 6
```

2. 산술·대입·증감 연산자

예제 4-5 증감 연산자 사용하여 연산하기 1

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 5, b = 5;
06     int re1 = 0, re2 = 0;
07
08     printf("초기상태 a: %d, b: %d\n", a, b);
09     printf("초기상태 re1: %d, re2: %d\n", re1, re2);
10
11     a++;
12     re1 = a;
13     re2 = b++;
14
15     printf("연산 후 결과 a: %d, b: %d\n", a, b);
16     printf("연산 후 결과 re1: %d, re2: %d\n", re1, re2);
17
18     return 0;
19 }
```

a의 값을 나중에 1 증가시키는 후위 연산

연산 후 a의 값(6)을 re1에 대입

b의 값(5)을 re2에 대입한 후 b의 값을 1 증가시킴

```
초기상태 a: 5, b: 5
초기상태 re1: 0, re2: 0
연산 후 결과 a: 6, b: 6
연산 후 결과 re1: 6, re2: 5
```

2. 산술·대입·증감 연산자

예제 4-6 증감 연산자 사용하여 연산하기 2

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 5, b = 5;
06     int re1 = 0, re2 = 0;
07
08     printf("초기상태 a: %d, b: %d\n", a, b);
09     printf("초기상태 re1: %d, re2: %d\n", re1, re2);
10
11     ++a;
12     re1 = a;
13     re2 = b++; // b의 값을 re2에 대입한 후 b의 값을 1 증가시킴
14
15     printf("연산 후 결과 a: %d, b: %d\n", a, b);
16     printf("연산 후 결과 re1: %d, re2: %d\n", re1, re2);
17
18     return 0;
19 }
```

a의 값을 1 증가시키는 전위 연산

연산 후 a의 값(6)을 re1에 대입

초기상태 a: 5, b: 5

초기상태 re1: 0, re2: 0

연산 후 결과 a: 6, b: 6

연산 후 결과 re1: 6, re2: 5

2. 산술·대입·증감 연산자

예제 4-7 이항 연산자와 증감 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 5;    // 5로 초기화
06     int b = 3;    // 3으로 초기화
07     int result;
08
09     result = a * b + (++a);    // 이항 연산자 *와 증감 연산자 ++ 사용
10     printf("결과 = %d\n", result);    6(증가된 a의 값) * 3 + 6(증가된 a의 값) = 24
11
12     int c = 6;    // 6으로 초기화
13     int d = 4;    // 4로 초기화
14
15     result = c * d + (c--);    // 이항 연산자 *와 감소 연산자 -- 사용
16     printf("결과 = %d\n", result);    6(현재 c의 값) * 4 + 6(현재 c의 값) = 30
17
18     return 0;
19 }
```

결과 = 24

결과 = 30

2. 산술·대입·증감 연산자

확인문제

1 다음 코드의 실행 결과값은 무엇인가?

```
int a = 10, b = 5, c = 6;  
result = ++a * --b + (++c);
```

Section 4.3

논리·조건·관계 연산자

3. 논리·조건·관계 연산자

■ 논리 연산자

- 특정 조건을 참(true) 또는 거짓(false)으로 결정하기 위한 연산자
- 2개 이상의 조건을 결합하거나 조건의 참, 거짓 여부를 반 전하는 등 논리 연산을 수행할 때 사용

표 4-4 논리 연산자의 종류

논리 연산자		설명	예
AND 연산자	&&	x와 y 두 조건 모두 참이면 결과가 참이다.	x && y
OR 연산자		x와 y 두 조건 중 하나라도 참이면 결과가 참이다.	x y
NOT 연산자	!	주어진 조건의 반댓값을 반환한다. 즉 조건이 참이면 거짓을 반환하고, 조건이 거짓이면 참을 반환한다.	!x

표 4-5 논리 연산자의 진리표

X	Y	X && Y	X Y	!X	!(X && Y)	!(X Y)
0	0	0	0	1	1	1
0	1	0	1	1	1	0
1	0	0	1	0	1	0
1	1	1	1	0	0	0

3. 논리·조건·관계 연산자

■ 논리 연산자

▪ AND 연산자(&&)

X = 15, Y = 5, W = 6, Z = 16일 때

(X > Y) && (W < Z)의 결과로 1을 반환

참

참

▪ OR 연산자(||)

X = 15, Y = 5, W = 6, Z = 16일 때

(X > Y) || (W > Z)의 결과로 1을 반환

참

거짓

▪ NOT 연산자(!)

X = 15, Y = 5일 때

!(X > Y)의 결과로 0을 반환

참

3. 논리·조건·관계 연산자

예제 4-8 논리 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 52, b = 31, c = 15;
06     int res1, res2, res3;    // 결과값을 저장할 변수 선언
07
08     res1 = a < b && b < c;    AND 연산자: 두 조건이 모두 참이면 참
09     printf("a는 b보다 작고, b는 c보다 작습니다.\n");
10     printf("a < b && b < c -> %d\n", res1);
11
12     res2 = a < b || a > c;    OR 연산자: 두 조건 중 하나라도 참이면 참
13     printf("a는 b보다 작거나, a는 c보다 큼니다.\n");
14     printf("a < b || a > c -> %d\n", res2);
15
16     res3 = !(a < b);         NOT 연산자: 결과의 반댓값 반환
17     printf("a는 b보다 작습니다.\n");
18     printf("!(a < b) -> %d\n", res3);
19
20     return 0;
21 }
```

```
a는 b보다 작고, b는 c보다 작습니다.
a < b && b < c -> 0
a는 b보다 작거나, a는 c보다 큼니다.
a < b || a > c -> 1
a는 b보다 작습니다.
!(a < b) -> 1
```

3. 논리·조건·관계 연산자

■ 조건 연산자

- 특정 조건에 따라 두 가지 표현식 중 하나를 선택하는 데 사용

| Syntax | 조건 연산자

조건 ? 표현식1 : 표현식2

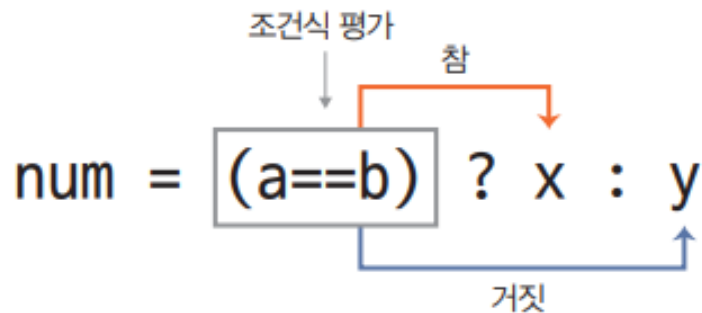
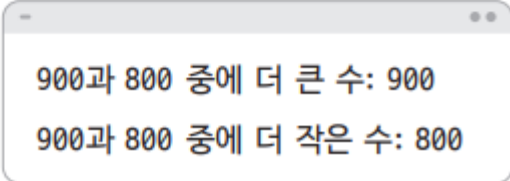


그림 4-5 조건 연산자의 동작 방식

3. 논리·조건·관계 연산자

예제 4-9 조건 연산자를 사용하여 큰 수 찾기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int maxV, minV;
06     int a = 900, b = 800;
07
08     maxV = (a > b) ? a : b; // a가 b보다 크면 maxV에 a 값을 대입, 그렇지 않으면 b 값을 대입
09     minV = (a < b) ? a : b; // a가 b보다 작으면 minV에 a 값을 대입, 그렇지 않으면 b 값을 대입
10
11     printf("900과 800 중에 더 큰 수: %d\n", maxV);
12     printf("900과 800 중에 더 작은 수: %d\n", minV);
13
14     return 0;
15 }
```



900과 800 중에 더 큰 수: 900
900과 800 중에 더 작은 수: 800

3. 논리·조건·관계 연산자

■ 관계 연산자

- 두 값을 비교하여 그 결과가 참(true)인지 거짓(false)인지 판단하기 위한 연산자
- 프로그램의 흐름을 제어하는 데 중요한 역할

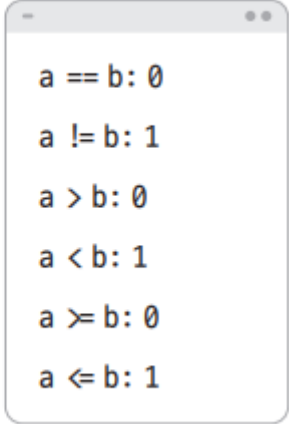
표 4-6 관계 연산자의 종류

관계 연산자	설명	예
==	두 값이 같다.	a == b(a와 b가 같으면 참)
!=	두 값이 같지 않다.	a != b(a와 b가 다르면 참)
>	왼쪽 값이 오른쪽 값보다 크다.	a > b(a가 b보다 크면 참)
<	왼쪽 값이 오른쪽 값보다 작다.	a < b(a가 b보다 작으면 참)
>=	왼쪽 값이 오른쪽 값보다 크거나 같다.	a >= b(a가 b보다 크거나 같으면 참)
<=	왼쪽 값이 오른쪽 값보다 작거나 같다.	a <= b(a가 b보다 작거나 같으면 참)

3. 논리·조건·관계 연산자

예제 4-10 관계 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 10;
06     int b = 20;
07
08     printf("a == b: %d\n", a == b); // a와 b가 같은지 확인
09     printf("a != b: %d\n", a != b); // a와 b가 다른지 확인
10     printf("a > b: %d\n", a > b); // a가 b보다 큰지 확인
11     printf("a < b: %d\n", a < b); // a가 b보다 작은지 확인
12     printf("a >= b: %d\n", a >= b); // a가 b보다 크거나 같은지 확인
13     printf("a <= b: %d\n", a <= b); // a가 b보다 작거나 같은지 확인
14
15     return 0;
16 }
```



```
a == b: 0
a != b: 1
a > b: 0
a < b: 1
a >= b: 0
a <= b: 1
```

3. 논리·조건·관계 연산자

예제 4-11 관계 연산자로 부동 소수점 수 비교하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     float a = 0.1f;
06     float b = 0.1f + 0.2f - 0.2f;
07
08     // 직접 비교
09     printf("a와 b는 %s\n", (a == b) ? "같습니다." : "같지 않습니다.");
10
11     // 오차 범위를 설정하여 비교
12     float epsilon = 0.000001;
13     printf("지정된 오차 범위 내에서 a와 b는 %s\n", ((a - b) < epsilon && (b - a) < epsilon)
14           ? "같습니다." : "같지 않습니다.");
15     return 0;
16 }
```

float형 변수 a를 0.1로 초기화

수식 계산을 이용한 초기화

a와 b의 값이 같은지 비교

오차 범위 epsilon 선언

a와 b의 차이가 허용 오차 범위보다 작은지 확인

a와 b는 같지 않습니다.

지정된 오차 범위 내에서 a와 b는 같습니다.

3. 논리·조건·관계 연산자

확인문제

1 논리 연산자와 기호를 바르게 연결하시오.

- | | |
|-------------|--------|
| ① AND 연산자 • | • ㉠ |
| ② OR 연산자 • | • ㉡ ! |
| ③ NOT 연산자 • | • ㉢ && |

2 관계 연산자의 의미와 기호를 바르게 연결하시오.

- | | |
|---------------------------|--------|
| ① 두 값이 같지 않다. • | • ㉠ == |
| ② 왼쪽 값이 오른쪽 값보다 크거나 같다. • | • ㉡ != |
| ③ 두 값이 같다. • | • ㉢ >= |
| ④ 왼쪽 값이 오른쪽 값보다 작거나 같다. • | • ㉣ <= |

3. 논리·조건·관계 연산자

LAB 학점 판별하기

■ 실행 결과

```
학생의 점수를 입력하세요: 96
학점: A

학생의 점수를 입력하세요: 88
학점: B

학생의 점수를 입력하세요: 55
학점: F
```



■ 조건

- 정수 범위의 점수를 입력받는다.
- 학점 판별 조건에는 중첩된 삼항 연산자와 관계 연산자를 사용한다.
- 점수별 학점 기준은 90점 이상은 A 학점, 80~89점은 B 학점, 70~79점은 C 학점, 60~69점은 D 학점, 60점 미만은 F 학점이다.

3. 논리·조건·관계 연산자

LAB 학점 판별하기

문제 해결

LAB_4-1.c

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int score;
07     char grade;
08
09     // 사용자로부터 점수 입력받기
10     printf("학생의 점수를 입력하세요: ");
11     scanf("%d", &score);
12
13     // 중첩된 삼항 연산자를 사용하여 학점 판별
14     grade = (score >= 90) ? 'A' :
15             (score >= 80) ? 'B' :
16             (score >= 70) ? 'C' :
17             (score >= 60) ? 'D' : 'F';
18
19     printf("학점: %c\n", grade);
20
21     return 0;
22 }
```

Section 4.4

기타 연산자

4. 기타 연산자

■ 비트 연산자

- 데이터를 비트 단위로 계산하여 연산을 수행할 때 사용하는 연산자
- 비트 연산자는 컴퓨터 하드웨어에 가까운 저수준 프로그래밍 작업인 하드웨어 제어, 메모리와 성능 최적화, 시스템 프로그래밍 등 에 주로 사용
- 문자 자료형을 비롯해 정수 계산이 가능한 정수 자료형(char, short, int, long 등)에서만 동작

표 4-7 비트 연산자의 종류

비트 연산자		설명
비트 논리 연산자	$\sim a$	a의 보수 연산(NOT)
	$a \& b$	a와 b의 비트 단위 논리곱(AND)
	$a b$	a와 b의 비트 단위 논리합(OR)
	$a \wedge b$	a와 b의 비트 단위 배타적 논리합(XOR)
비트 시프트 연산자	$a \ll c$	a의 각 비트를 왼쪽으로 c만큼 이동
	$a \gg c$	a의 각 비트를 오른쪽으로 c만큼 이동

4. 기타 연산자

■ 비트 연산자

■ 비트 보수 연산자(~)

↓ 부호 비트 반전

$c = \sim a$	$\sim$	$a = 00000000\ 00001010$	$a = 10$
		$c = 11111111\ 11110101$	$c = -11$

그림 4-6 $c = \sim a$ 의 내부적 비트 연산 과정

■ 비트 논리곱 연산자(&)

$a \ \& \ b$		$a = 00000000\ 00001010$	$a = 10$
	$\&$	$b = 00000000\ 00000101$	$b = 5$
		$c = 00000000\ 00000000$	$c = 0$

그림 4-7 $a \ \& \ b$ 의 내부적 비트 연산 과정

4. 기타 연산자

■ 비트 연산자

■ 비트 논리합 연산자(|)

a b	a = 00000000 00001010	a = 10
	b = 00000000 00000101	b = 5
	c = 00000000 00001111	c = 15

그림 4-8 a | b의 내부적 비트 연산 과정

■ 비트 배타적 논리합 연산자(^)

a ^ b	a = 00000000 00001010	a = 10
	b = 00000000 00000101	b = 5
	c = 00000000 00001111	c = 15

그림 4-9 a ^ b의 내부적 비트 연산 과정

4. 기타 연산자

예제 4-12 비트 논리 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 0x5555, b = 0x00ff;
06
07     printf("a = %x, b = %x\n", a, b);
08     printf("~a = %x, ~b = %x\n", ~a, ~b);    // 비트 NOT 연산
09     printf("a & b = %x\n", a & b);           // 비트 AND 연산
10     printf("a | b = %x\n", a | b);           // 비트 OR 연산
11     printf("a ^ b = %x\n", a ^ b);           // 비트 XOR 연산
12
13     return 0;
14 }
```

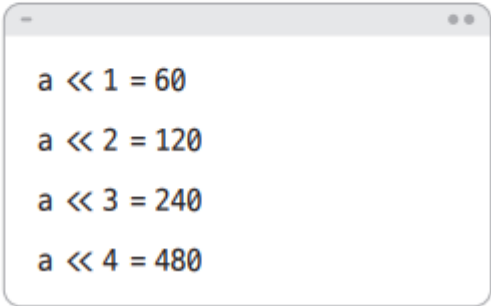
변숫값을 16진수로 초기화

```
a = 5555, b = ff
~a = fffffaaa, ~b = ffffffff00
a & b = 55
a | b = 55ff
a ^ b = 55aa
```

4. 기타 연산자

예제 4-13 비트 시프트 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 30;
06
07     // 왼쪽으로 1비트씩 이동할 때마다 값이 2배가 됨
08     printf("a << 1 = %d\n", a << 1);
09     printf("a << 2 = %d\n", a << 2);
10     printf("a << 3 = %d\n", a << 3);
11     printf("a << 4 = %d\n", a << 4);
12
13     return 0;
14 }
```



```
a << 1 = 60
a << 2 = 120
a << 3 = 240
a << 4 = 480
```

4. 기타 연산자

■ 주소 연산자와 주소 참조 연산자

■ 주소 연산자(&)

| Syntax | 주소 연산자 &

&변수명

&x

■ 주소 참조 연산자(*)

| Syntax | 주소 참조 연산자 *

자료형 *변수명

int *x

```
int x = 10;
```

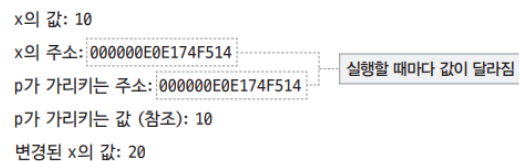
```
int *p = &x;      // p는 x의 주소를 저장
```

```
printf("%d", *p);  // p가 가리키는 변수의 값인 10을 출력
```

4. 기타 연산자

예제 4-14 주소 참조 연산자 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int x = 10;    // 정수 변수
06     int *p;        // 정수 포인터 선언
07
08     p = &x;        변수 x의 메모리 주소를 반환한 후 p에 대입
09
10     printf("x의 값: %d\n", x);
11     printf("x의 주소: %p\n", &x);
12     printf("p가 가리키는 주소: %p\n", p);
13     printf("p가 가리키는 값 (참조): %d\n", *p);
14
15     // p를 통해 x의 값을 변경
16     *p = 20;        포인터 p가 가리키는 메모리 위치의 값 반환
17
18     printf("변경된 x의 값: %d\n", x);
19
20     return 0;
21 }
```



x의 값: 10
x의 주소: 000000E0E174F514
p가 가리키는 주소: 000000E0E174F514
p가 가리키는 값 (참조): 10
변경된 x의 값: 20

실행할 때마다 값이 달라짐

4. 기타 연산자

확인문제

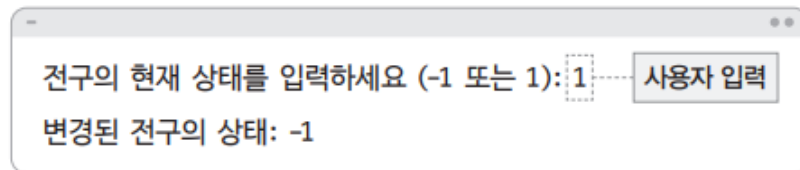
1 다음 빈칸에 알맞은 말을 넣으시오.

- ① ()는 주어진 숫자의 각 비트를 반전하므로 0은 1이 되고 1은 0이 된다.
- ② ()는 두 2진수의 각 비트를 비교하여 둘 다 1이면 1을 반환하고, 둘 중 하나라도 1이 아니면 0을 반환한다.
- ③ ()는 두 2진수의 각 비트를 비교하여 둘 중 하나라도 1이면 1을 반환하고, 둘 다 0이면 0을 반환한다.
- ④ ()는 변수의 비트를 지정된 수만큼 왼쪽으로 이동한다.
- ⑤ ()는 변수의 비트를 지정된 수만큼 오른쪽으로 이동한다.

4. 기타 연산자

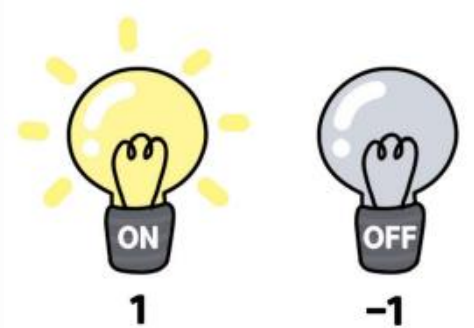
LAB 전구 상태 변경하기

■ 실행 결과



전구의 현재 상태를 입력하세요 (-1 또는 1): 1 사용자 입력

변경된 전구의 상태: -1



■ 조건

- 전구의 현재 상태를 - 1(꺼진 상태) 또는 1(켜진 상태)로 입력받는다.
- ~ 연산자를 사용하여 전구의 상태를 변경한다.
- 변경된 전구의 상태를 출력한다.

■ 힌트

- 1의 보수는 ~ 연산자를 사용하여 얻을 수 있다.
- 1의 보수를 구한 후 1을 더하면 2의 보수를 얻을 수 있다.

4. 기타 연산자

LAB 전구 상태 변경하기

문제 해결

LAB_4-2.c

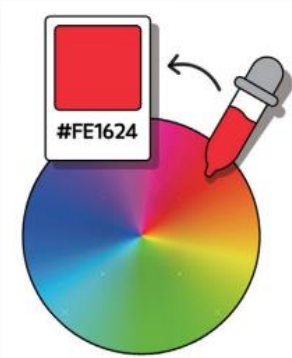
```
01 #pragma warning(disable:4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int bulbState;
07
08     printf("전구의 현재 상태를 입력하세요 (-1 또는 1): ");
09     scanf("%d", &bulbState);
10
11     bulbState = ~bulbState + 1;
12
13     printf("변경된 전구의 상태: %d\n", bulbState);
14
15     return 0;
16 }
```

4. 기타 연산자

LAB RGB 색상 생성하기

■ 실행 결과

```
빨강(Red) 값 입력 (0~255): 254
초록(Green) 값 입력 (0~255): 22
파랑(Blue) 값 입력 (0~255): 36
입력된 RGB 값에 대한 색상 코드: #FE1624
```



■ 조건

- 빨강(Red), 초록(Green), 파랑(Blue) 값을 0~255의 정수로 입력받는다.
- RGB 색상은 24비트로 표현된다. 비트 시프트 연산자를 사용하여 각 색상의 값을 올바른 위치에 배치하고, 비트 OR 연산자를 사용하여 조합한다.
- printf() 함수에서 %06X 서식 지정자를 사용하여 색상 코드를 16진수로 출력한다.

4. 기타 연산자

LAB RGB 색상 생성하기

■ 힌트

- `red << 16`: 빨강(Red) 값을 왼쪽으로 16비트 이동한다. 216을 계산하는 것과 같으며, 빨강 값이 색상 코드의 왼쪽에 위치한다.
- `green << 8`: 초록(Green) 값을 왼쪽으로 8비트 이동한다. 28을 계산하는 것과 같으며, 초록 값이 색상 코드의 중간에 위치한다.
- `blue`: 파랑(Blue) 값은 그대로 사용되며, 색상 코드의 오른쪽에 위치한다.
- `|`: 비트 OR 연산자로 세 가지 값을 결합하며 빨강, 초록, 파랑의 각 비트가 모두 합쳐져서 하나의 24비트 색상 코드가 생성된다.
- `%X`: 16진수를 출력하는 서식 지정자를 사용한다. `%06X`는 출력될 16진수의 최소 자릿수를 여섯 자리로 지정한다. 여섯 자리보다 짧으면 앞에 0으로 채워지므로 출력 결과는 `#RRGGBB` 형태가 된다.

4. 기타 연산자

LAB RGB 색상 생성하기

문제 해결

LAB_4-3.c

```
01 #pragma warning(disable: 4996)
02 #include <stdio.h>
03
04 int main()
05 {
06     int red, green, blue;
07
08     printf("빨강(Red) 값 입력 (0-255): ");
09     scanf("%d", &red);
10     printf("초록(Green) 값 입력 (0-255): ");
11     scanf("%d", &green);
12     printf("파랑(Blue) 값 입력 (0-255): ");
13     scanf("%d", &blue);
14
15     int colorCode = (red << 16) | (green << 8) | blue;
16     printf("입력된 RGB 값에 대한 색상 코드: %#06X\n", colorCode);
17
18     return 0;
19 }
```

RGB 값을 입력받음

시프트 연산자와 비트 OR 연산자를 사용하여 하나의 24비트 색상 코드 생성

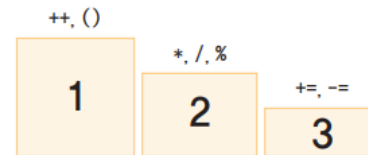
생성된 24비트 색상 코드를 16진수로 출력

Section 4.5

연산자의 우선순위

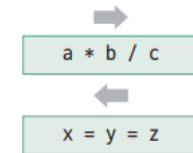
5. 연산자의 우선순위

■ 연산자의 우선순위와 결합 방향



우선순위가 높은 연산자 먼저 연산

(a) 연산자의 우선순위가 다른 경우



결합 방향에 따른 우선순위

(b) 연산자의 우선순위가 같은 경우

표 4-8 연산자의 우선순위와 결합 방향

우선순위	연산자	설명	결합 방향
1	p++, p--	후위 증감 연산자	→
	()	괄호, 함수 호출	
	[]	배열 첨자	
	.	구조체	
	->	포인터로 구조체 접근	
	(자료형){값}	복합 리터럴	
2	++p, --p	전위 증감 연산자	←
	+p, -p	양의 부호, 음의 부호	
	!	논리 NOT	
	~	비트 NOT	
	(자료형)	자료형 변환	
	*p	포인터 p 역참조	
	&x	x의 주소	
	sizeof	자료형의 크기	
3	*, / , %	곱셈, 나눗셈, 나머지	→
4	+, -	덧셈, 뺄셈	→
5	<<	비트를 왼쪽으로 이동	→
	>>	비트를 오른쪽으로 이동	

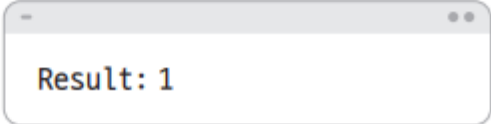
그림 4-12 연산자의 우선순위

우선순위	연산자	설명	결합 방향
6	<	작음	→
	<=	작거나 같음	
	>	큼	
	>=	크거나 같음	
7	==	같음	→
	!=	다름	
8	&	비트 AND	→
9	^	비트 XOR	→
10		비트 OR	→
11	&&	논리 AND	→
12		논리 OR	→
13	?, :	조건(삼항) 연산자	←
14	=	대입 연산자	←
	+=, -=	복합 대입 연산자	
	*=, /=, %=	복합 대입 연산자	
	<<=, >>=	복합 대입 연산자	
	&=, ^=, =	복합 대입 연산자	
15	,	coma 연산자	→

5. 연산자의 우선순위

예제 4-15 연산자의 우선순위 사용하기

```
01 #include <stdio.h>
02
03 int main()
04 {
05     int a = 5;
06     int b = 10;
07     int c = 5;
08
09     // == 연산자와 > 연산자는 && 연산자보다 우선순위가 높다.
10     int result = a == c && b > a;
11     printf("Result: %d\n", result);    // 출력: 1 (true)
12
13     return 0;
14 }
```



Result: 1

5. 연산자의 우선순위

확인문제

1 연산자의 우선순위에 대한 설명 중 옳지 않은 것을 고르시오.

- ① 연산자의 우선순위가 높을수록 연산이 먼저 수행된다.
- ② 여러 개의 연산자를 사용할 때 결합 방향은 왼쪽에서 오른쪽으로 모두 동일하다.
- ③ ++a, --a 연산자가 a++, a-- 연산자보다 먼저 수행된다.
- ④ 연산자의 우선순위가 같은 경우 결합 방향에 따라 순서가 결정된다.

2주차 강의가 끝났습니다,
모두 고생하셨습니다.

